SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

GURRAM SAIRAM(VTU25501)

10211CS224
Full Stack Application Development
Slot : E1

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”Smart Campus Event Management System” by G.SaiRam (VTU25501) has been carried out in Winter semester of Academic year 2025- 2026(WS2526).

Signature of Faculty

Dr.S.Hemamalini

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinater

Dr.M.Sumithra

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(G.SaiRam)

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management System is a comprehensive full-stack web application designed to streamline the organization, discovery, and registration of campus wide events. Built using a modern Spring Boot backend and a responsive React frontend, the system bridges the communication gap between event organizers Admins and participants Students.

The platform features a secure Admin Dashboard that allows authorized personnel to create, update, and monitor various campus events ranging from technical workshops to cultural festivals. Students benefit from an intuitive Events Explorer interface where they can search for opportunities by department or category. A key innovation of the system is its secure registration workflow, which utilizes automated Email Notifications to verify student identities and provide instant confirmation. The application leverages a persistent Mysql database to ensure data integrity and features a premium, responsive design tailored for a superior user experience across all devices.

Keywords: Smart Campus, Event Management System, Online Registration, Centralized Platform, Student-Admin Interaction, Event Scheduling, Notifications, Participant Management, Automation, Campus Events,Secure Authentication, Data Persistence, Real-time Updates, User Experience, Cross-platform Accessibility, Workshop Coordination.

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
EMS	Event Management System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
JS	JavaScript
MVC	Model
	View Controller

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

In the modern educational landscape, a vibrant campus life is defined by the diversity and frequency of its events ranging from academic seminars and technical workshops to cultural festivals and sports competitions. As campuses grow larger and more dynamic, the traditional methods of managing these events through physical notices, scattered emails, or manual registration forms have become increasingly inefficient. The Smart Campus Event Management System is a digital solution designed to centralize and automate this entire lifecycle, providing a unified platform for both administrators and students.

1.2 Aim of the Project

The primary aim of this project is to design and develop a centralized, automated, and secure digital platform for managing campus events. The system intends to replace manual coordination with a modern web-based solution that simplifies event discovery for students and streamlines administrative tasks for organizers, ultimately fostering a more connected and engaged campus environment.

1.3 Scope of the Project

The scope of the project includes the design and implementation of a comprehensive web-based platform that manages the entire lifecycle of campus events, from creation to student participation. It covers the development of an administrator module for scheduling events and managing capacities, alongside a student-facing portal for searching and exploring upcoming workshops, seminars, and festivals. The system also integrates a secure registration workflow that utilizes automated email-based QR verification and instant confirmation messaging to ensure data integrity and student authenticity.

1.4 Framework

The project utilizes a cohesive Full-Stack Framework architecture that integrates a Spring Boot backend with a React frontend to deliver a high-performance, scalable web application. The backend leverages the Spring ecosystem, specifically using Spring MVC for request handling and Spring Data JPA for database management, providing a secure and robust foundation for the application's business logic. Complementing this, the React frontend employs a component-based architecture to create a dynamic, single-page experience that interacts with the backend via RESTful API calls. This architectural synergy is supported by the Maven build tool for backend dependency management and the Vite development server for frontend optimization, ensuring a streamlined development lifecycle and a modern, responsive user interface.

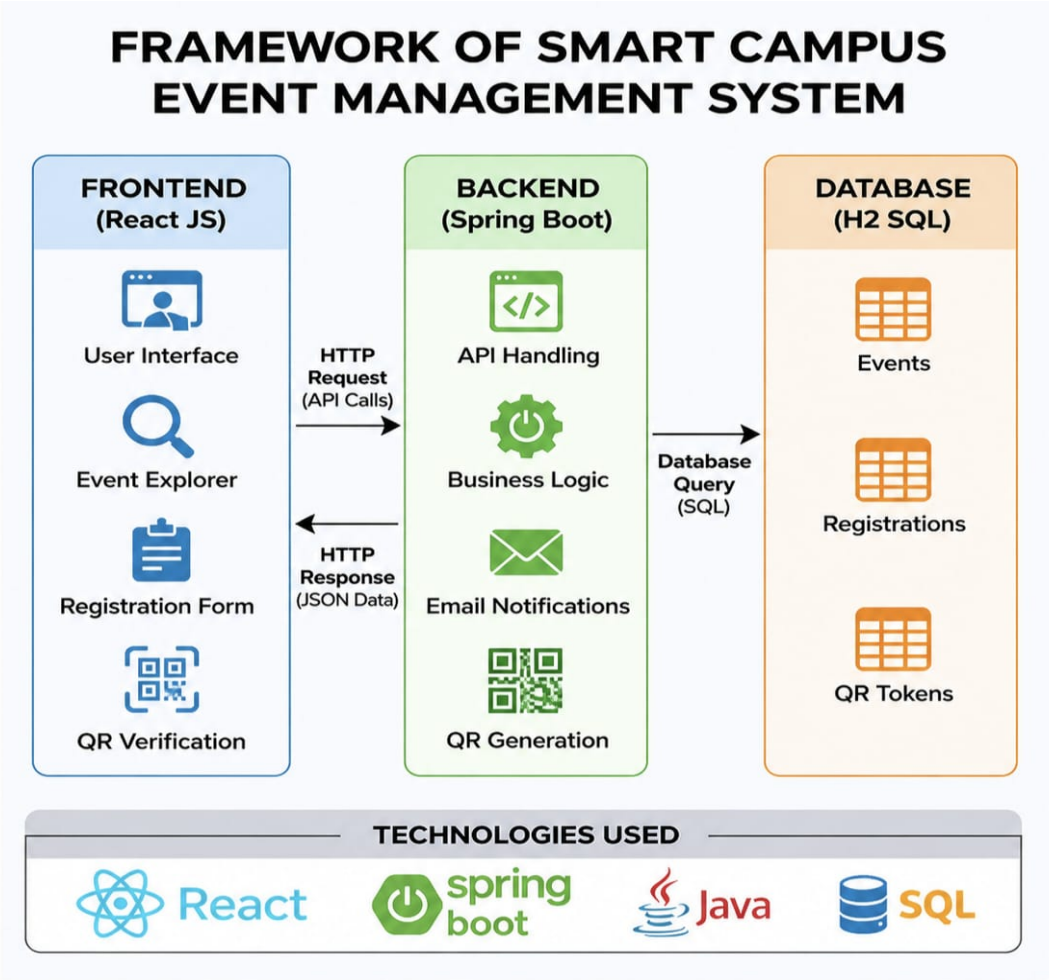

Figure 1.1: Framework of smart campus event management

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

In the current market, several event management and registration platforms have been developed to simplify the discovery and reservation process for large scale activities. One of the most prominent platforms is BookMyShow, which provides specialized services for booking tickets for movies, concerts, and sports events. It offers features such as centralized event listings, secure verification, and user-friendly navigation, making it a benchmark for consumer-facing event platforms. Similarly, global platforms like Eventbrite and Ticketmaster provide large scale solutions with secure transactions, event promotion, and real-time participant management, catering to millions of users worldwide.

This survey highlights that while current market applications are highly functional and scalable for public events, there is a significant gap in cost-effective, simplified solutions tailored for smaller, closed environments like educational institutions. The proposed Smart Campus Event Management System addresses these needs by providing a lightweight, user-friendly, and efficient platform.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is configured using Node.js and npm for managing dependencies. The user interface is built using React 19, leveraging JSX for component structure and Vanilla CSS for premium styling. The build process is powered by Vite, ensuring high speed development and optimized production bundles. Visual Studio Code serves as the primary IDE, while the application is tested and debugged using modern web browsers like Google Chrome.

3.1.2 Structure of the Project

The frontend follows a modular, component based architecture. It is divided into reusable components such as the Event Card, Registration Form, and Admin Dashboard. The project is organized into dedicated folders.

3.1.3 Environment Setup

To launch the frontend, dependencies are first installed via npm install. The development server is then started using the npm run dev command. Once active, the

React application communicates with the Spring Boot backend via REST API calls, allowing students to browse events and admins to manage records in real-time.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend environment is configured using the Spring Boot 3.2.0 framework, which provides a robust and scalable environment for handling academic data. The Java Development Kit (JDK 17+) is used for compiling and running the application logic. Unlike traditional setups, this project utilizes the Mysql Database engine in persistent file mode, which eliminates the need for a separate database server. The environment is managed using Apache Maven for dependency handling and the Eclipse IDE and VsCode for development and debugging.

3.2.2 Structure of the Project

The structure of the project follows a modular, full-stack architecture that clearly separates the frontend, backend, and database layers for improved maintainability and scalability. The frontend is developed using React JS, organized into reusable functional components such as event explorers, registration forms, and administrative panels, with styling managed through centralized CSS files.

3.2.3 Execution Procedure

To run the application, the required frontend dependencies are first installed using npm, and the React development server is started to launch the user interface. Simultaneously, the backend is executed by starting the Spring Boot application through Eclipse or the Maven wrapper, which initializes the embedded Tomcat server on

port 8089. The system automatically configures and connects to the persistent H2 database to manage event and registration data.

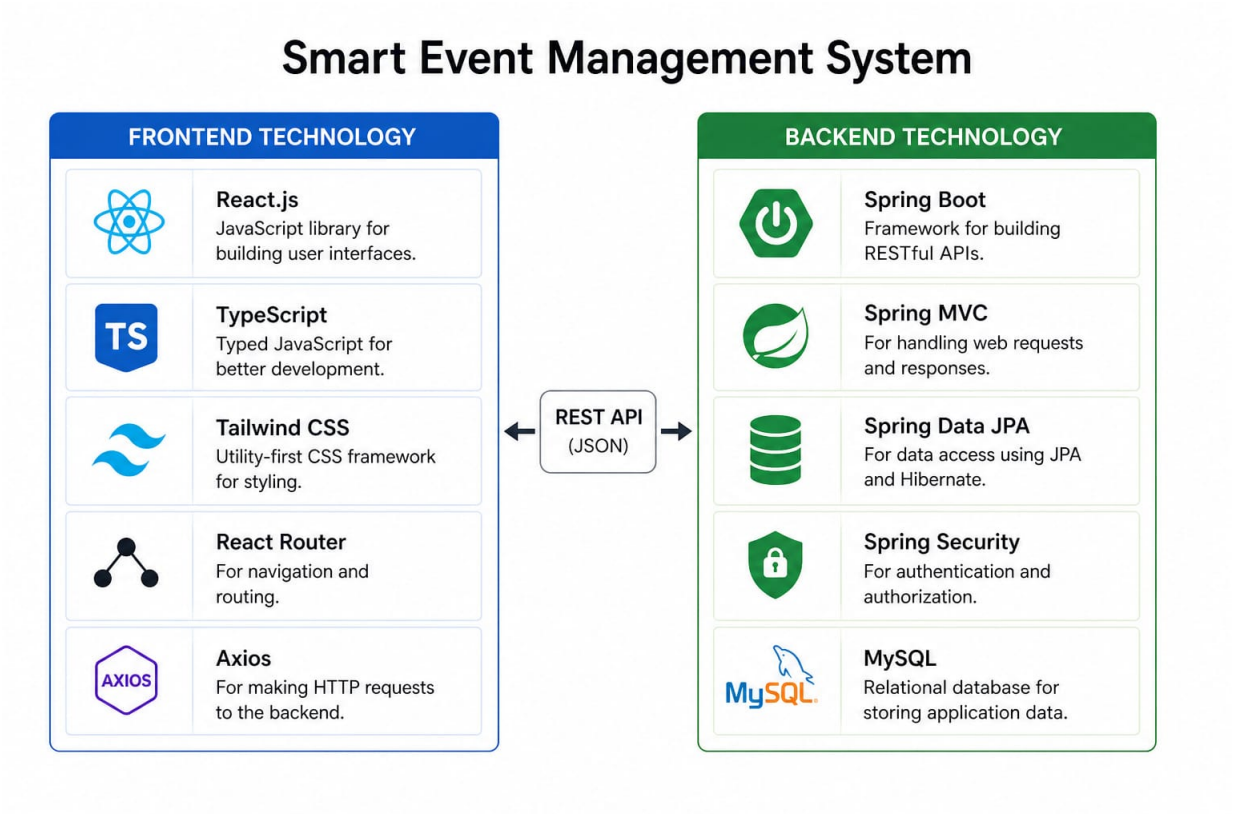

Figure 3.1: Frontend And Backend Architecture

3.3 Database Configuration

3.3.1 Database Configuration

The database is configured using MySQL, a high-performance relational database, to manage event and registration data. Unlike embedded databases, MySQL operates as a standalone database server, providing robust data storage, scalability, and multi-user access. It is integrated with the Spring Boot application through proper configuration, enabling efficient data retrieval, persistence, and management in a structured manner.

3.3.2 Schema Structure

The database schema is designed to efficiently handle event logistics and student participation records. It includes structured tables with fields for event details title, description, date, department, type, and capacity and registration details Like student name, email, student ID, and event reference. Relationships are maintained using Foreign Keys, linking each registration to its corresponding event via the event.

3.3.3 Tables Created

The database includes three main tables: the Events Table for storing event details like title, date, and capacity; the Registrations Table for keeping track of student names, emails, and IDs; and the QR Tokens Table for managing secure verification codes. These tables work together to ensure that all event information and student bookings are stored accurately and can be retrieved quickly.

Additionally, each table is designed with primary keys to uniquely identify records and maintain data integrity. Foreign key relationships are used to connect the Events and Registrations tables, ensuring proper linkage between events and participants. Indexing is applied on important fields like event ID and student ID to support fast data retrieval.

Data validation rules are implemented to prevent duplicate registrations and incorrect entries. The QR Tokens table stores unique codes generated for each registration, enabling secure and quick verification during event entry. Timestamps are maintained to track registration time and event updates.

The database structure follows normalization principles to reduce redundancy and improve efficiency. It is also scalable, allowing additional tables such as payment or feedback to be integrated in the future. Backup and recovery mechanisms can be implemented to ensure data safety and reliability.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a robust layered architecture with presentation, application, and database layers. The presentation layer uses React JS to provide a dynamic and responsive interface for students and admins. The application layer is built with Spring Boot, handling RESTful APIs, business logic, email generation, authentication, and secure communication. The database layer uses MySQL to store and manage user data, event details, and transaction records efficiently, ensuring data consistency and scalability. The architecture supports modular development, allowing each layer to be independently developed, tested, and maintained. This improves code reusability, simplifies debugging, and enables easier integration of new features without affecting existing system functionality. Advanced security measures such as role-based access control RBAC, data encryption, and secure API communication are implemented to safeguard sensitive user, event, and transaction data, ensuring privacy and protection against unauthorized access.

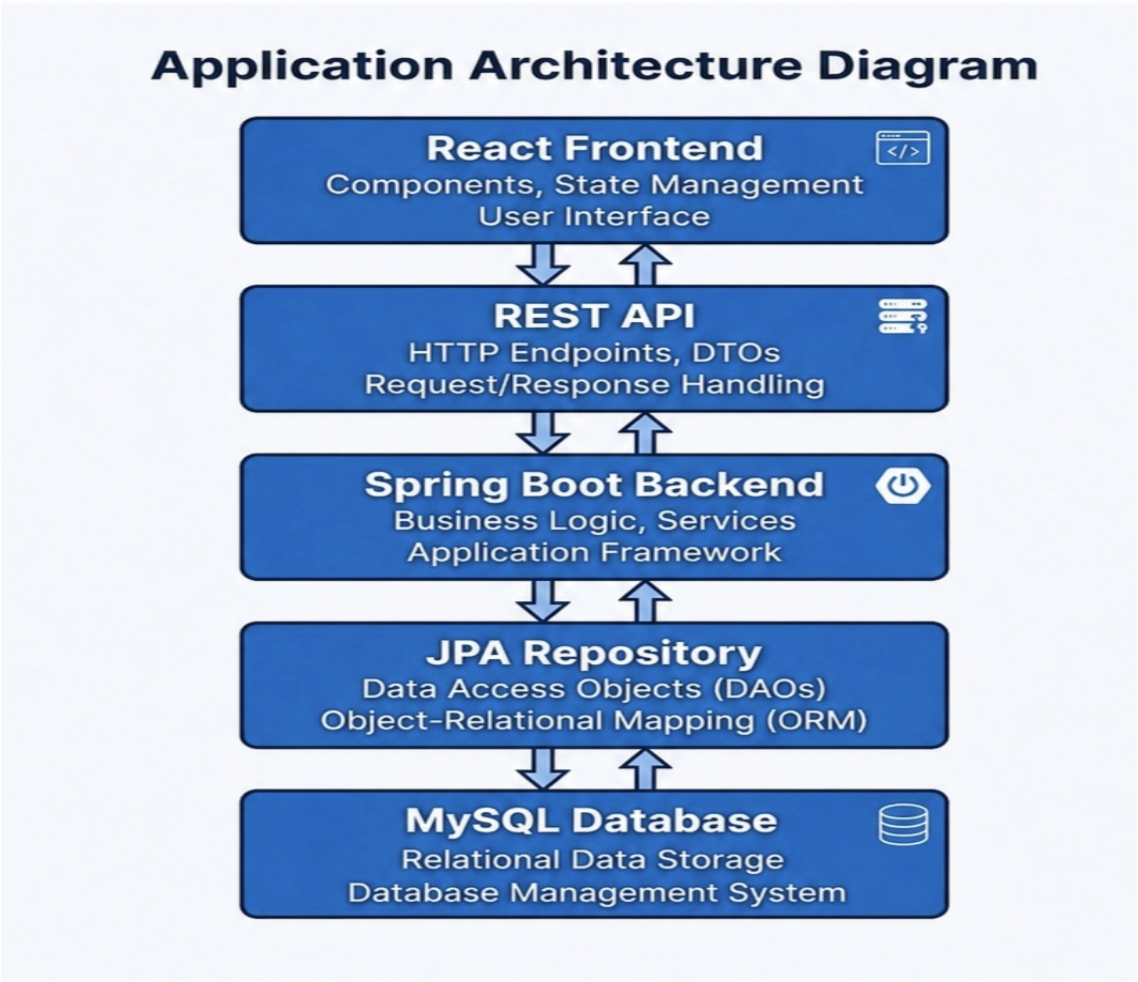

Figure 4.1: System Architecture Diagram

4.2 DataFlow Diagram

The data flow begins when a student explores events on the React frontend, which sends an API request to the Spring Boot backend to fetch real-time event data. When a student registers,which is generated by the backend and sent via email. Once the student submits the E-Mail along with their registration details, the backend validates the token, processes the enrollment, and stores the student's information in the Mysql database. Finally, the system updates the event capacity and sends a confirmation response back to the frontend to notify the student.

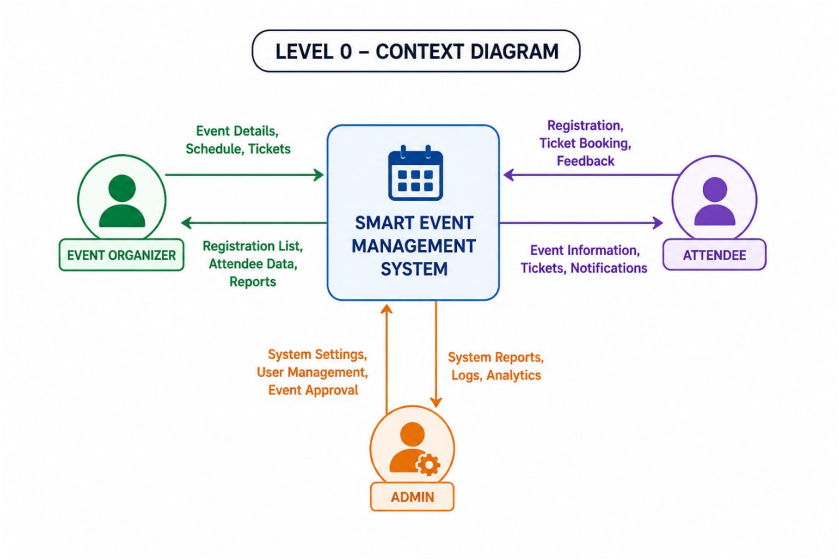

Figure 4.2: DataFlow Diagram

4.3 Module 1: User Interface Module

This module manages the student-facing interface. It includes the event explorer grid, search filters, and the registration portal. It is designed to provide a seamless user experience, allowing students to discover workshops or festivals and initiate the secure Email based registration process through an intuitive UI.

4.4 Module 2: API and Backend Module

This module is the core of the system, managing the communication between the user and the data. It includes the QR Generation Service, Email Notification System, and Admin Authentication. The backend ensures that all input data is validated and that only verified students can successfully register for campus events.

4.5 Module 3: Database Module

This module is responsible for administrative oversight and data integrity. It provides a secure dashboard for event creation and participant tracking. It interacts directly with the Mysql Database to ensure that event details and registration records are stored systematically and retrieved efficiently for administrative review.

Advantages

Secure Registration: The integration of Email verification ensures that only authorized students can register for events, preventing spam.

Automated Notifications: Students receive instant email confirmations, improving the overall communication flow.

Centralized Management: Provides a single platform for all campus activities, eliminating the need for scattered physical notices.

Lightweight Persistent: Uses an embedded Mysql database that is fast, efficient, and requires no complex server setup while maintaining data persistence.

Disadvantages

Scalability: While perfect for a campus environment, the system may require further optimization for university wide massive deployments.

Payment Integration: Currently, the system supports free event registrations and does not include a payment gateway for paid workshops.

Local Hosting: The system is currently optimized for local or internal campus network hosting rather than global cloud deployment.

Chapter 5

RESULTS AND DISCUSSIONS

The Smart Campus Event Management System was successfully implemented and tested for stability. The application allows students to view events and register securely, with the React frontend providing a smooth interface and the Spring Boot backend managing data in the Mysql database. Testing confirmed that all student registration details are correctly validated and stored, with automated emails sent upon successful booking.

Applications	**Advantages**
Event Registration System	Simplifies user enrollment for events
Event Scheduling	Helps organize and manage event timings efficiently
Online Ticketing	Enables quick and secure ticket booking
QR Code Verification	Provides fast and contactless entry at events
Notification System	Sends real-time updates and reminders
Event Analytics	Offers insights on participation and performance

Table 5.1: Application Comparison

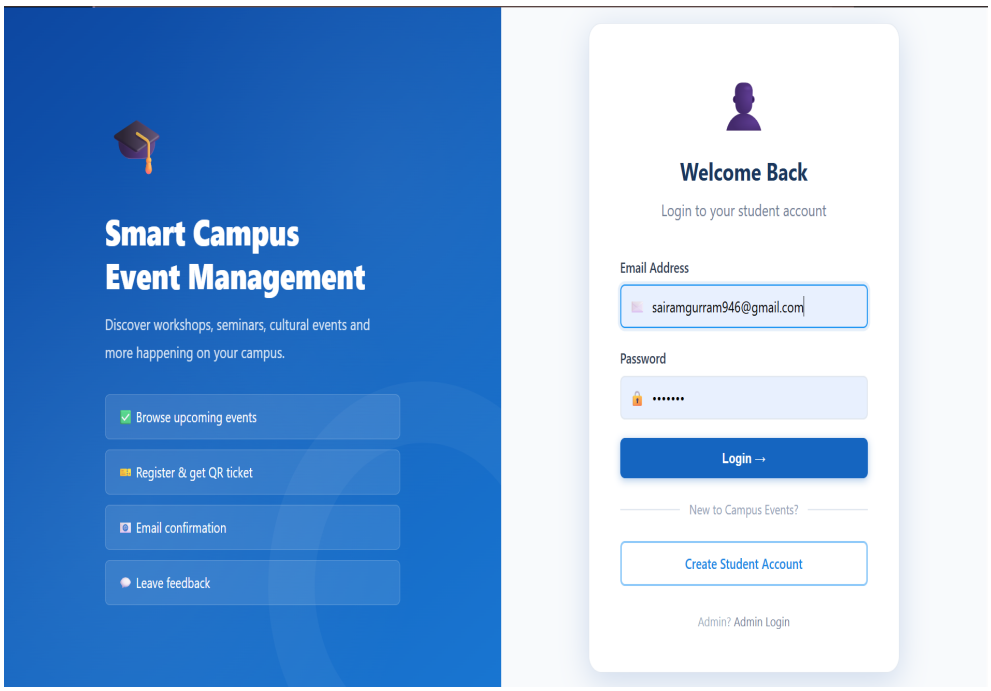

Figure 5.1: Login page for Student

The Smart Event Management System dashboard provides a centralized and user-friendly interface for managing all event-related activities efficiently. It allows users to explore upcoming events, view detailed schedules, and book tickets seamlessly with real-time confirmation and status updates. Attendees can access their registered events, download QR-based tickets, and receive instant notifications about event changes, reminders, and announcements.Event organizers can create and manage events, track registrations, monitor attendee data, and analyze participation through insightful reports.

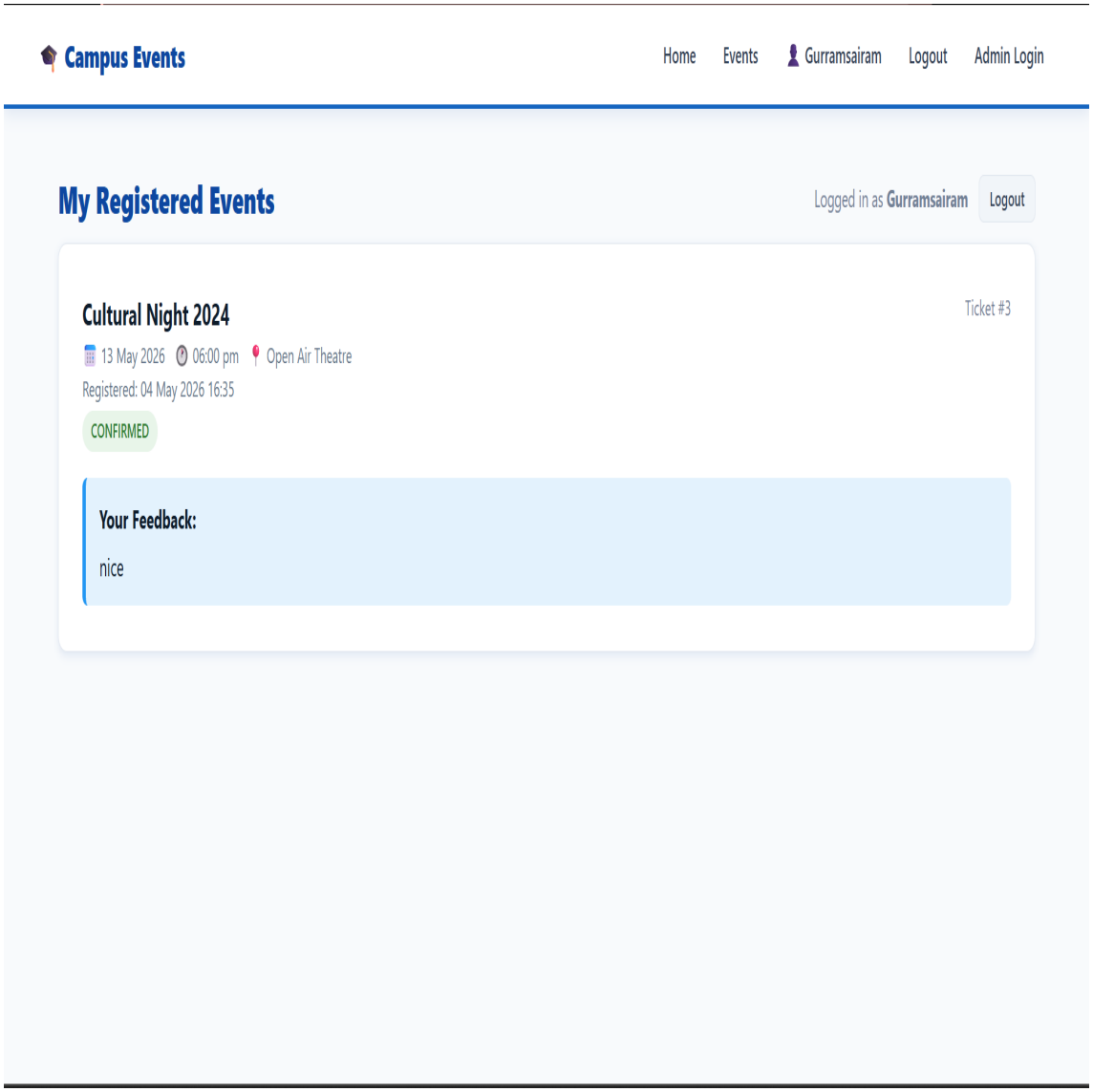

Figure 5.2: Student Dashboard

The admin organizer dashboard of the Smart Event Management System provides a comprehensive overview of all event management activities through a centralized and analytics-driven interface. It enables organizers to create, update, and manage events, monitor active and upcoming events, and track the total number of registrations and bookings in real time. The dashboard displays key insights such as attendee trends, ticket sales, event-wise participation, and revenue generated.Organizers can view detailed information about attendees, manage registrations, and control event capacity effectively.

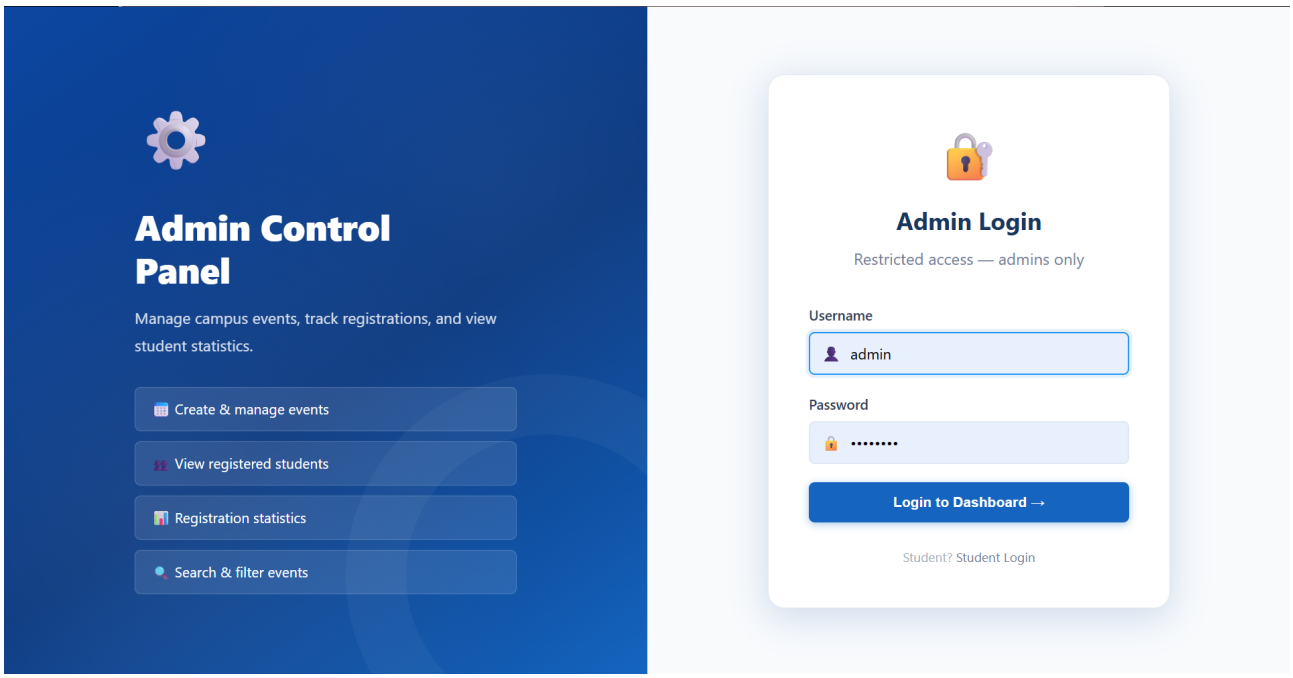

Figure 5.3: Admin/Employer Dashboard

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System successfully digitizes campus event logistics using a Spring Boot and React JS architecture. By automating registrations with secure verification and email notifications, the system reduces manual errors and improves communication between admins and students. The project achieves its goal of providing a reliable, user-friendly, and centralized platform for efficient campus activity management.

6.2 Future Enhancements

The system can be enhanced by integrating secure online payment gateways to support paid event registrations and automated refund management. A real-time analytics dashboard can be introduced to provide insights into user participation, popular events, and engagement trends. Advanced features such as AI-based personalized event recommendations and chatbot support can improve user interaction and help users discover relevant events.

Chapter 7

Addressing Complex Engineering Problem and SDG

7.1 Mapping to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth knowledge	WK3, WK4, WK5	Expertise in Spring Boot, React JS, and REST API architecture.
WP2	Conflicting Req.	Technical vs non-technical	WK4, WK5	Balances strict security (OTP) with a seamless UX.
WP3	Depth of Analysis	Analytical design thinking	WK3, WK4	System design requires analysis of registrations.
WP4	Familiarity	Partially new problems	WK4, WK5	Integrating persistent OTP verification across full-stack.
WP5	Applicable Codes	Limited standard solutions	WK6	Custom security design using persistent OTP tokens.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK7	Supports students, faculty, and administrators.
WP7	Interdependence	System level integration	WK3, WK6, WK5	Integrates frontend UI, backend services.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Diverse technologies	Spring Boot, React JS, Maven, H2 Database, SMTP APIs.
EA2	Level of Interactions	Complex module interactions	Seamless flow between Event Creation and OTP Verification.
EA3	Innovation	Modern technologies	Real-time student verification via secure persistent OTP.
EA4	Societal Impact	Impact on users	Improves accessibility to academic opportunities.
EA5	Familiarity	Beyond standard practice	Digitizing traditional campus event logistics.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Centralizes academic seminars and workshops.
SDG 9	Industry & Innovation	Implements digital infrastructure for campus.
SDG 11	Sustainable Cities	Reduces paper waste through digital event logistics.
SDG 16	Peace & Justice	Ensures transparency through secure OTP verification.
SDG 12	Responsible Consumption	Cuts paper use through online event registration.

Table 7.3: SDG Mapping for the Project

Chapter 8

SOURCE CODE

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
.counter {
  font-size: 16px;
  padding: 5px 10px;
  border-radius: 5px;
  color: var(--accent);
  background: var(--accent-bg);
  border: 2px solid transparent;
  transition: border-color 0.3s;
  margin-bottom: 24px;

  &:hover {
    border-color: var(--accent-border);
  }
  &:focus-visible {
```

```
    outline: 2px solid var(--accent);
    outline-offset: 2px;
  }
}

.hero {
  position: relative;

  .base,
  .framework,
  .vite {
    inset-inline: 0;
    margin: 0 auto;
  }

  .base {
    width: 170px;
    position: relative;
    z-index: 0;
  }

  .framework,
  .vite {
    position: absolute;
  }

  .framework {
    z-index: 1;
    top: 34px;
    height: 28px;
    transform: perspective(2000px) rotateZ(300deg) rotateX(44deg) rotateY(39deg)
      scale(1.4);
  }

  .vite {
    z-index: 0;
    top: 107px;
    height: 26px;
    width: auto;
```

```
    transform: perspective(2000px) rotateZ(300deg) rotateX(40deg) rotateY(39deg)
      scale(0.8);
  }
}

#center {
  display: flex;
  flex-direction: column;
  gap: 25px;
  place-content: center;
  place-items: center;
  flex-grow: 1;

  @media (max-width: 1024px) {
    padding: 32px 20px 24px;
    gap: 18px;
  }
}

#next-steps {
  display: flex;
  border-top: 1px solid var(--border);
  text-align: left;

  & > div {
    flex: 1 1 0;
    padding: 32px;
    @media (max-width: 1024px) {
      padding: 24px 20px;
    }
  }

  .icon {
    margin-bottom: 16px;
    width: 22px;
    height: 22px;
  }

  @media (max-width: 1024px) {
```

```
    flex-direction: column;
    text-align: center;
  }
}

#docs {
  border-right: 1px solid var(--border);

  @media (max-width: 1024px) {
    border-right: none;
    border-bottom: 1px solid var(--border);
  }
}

#next-steps ul {
  list-style: none;
  padding: 0;
  display: flex;
  gap: 8px;
  margin: 32px 0 0;

  .logo {
    height: 18px;
  }

  a {
    color: var(--text-h);
    font-size: 16px;
    border-radius: 6px;
    background: var(--social-bg);
    display: flex;
    padding: 6px 12px;
    align-items: center;
    gap: 8px;
    text-decoration: none;
    transition: box-shadow 0.3s;}
```

References

[1] Meta Platforms Inc. (2024). *React: A JavaScript Library for Building User Interfaces*. React Official Documentation. Available at: https://react.dev/

[2] Broadcom Inc. (2024). *Spring Boot: Build Anything with Spring*. Spring Official Documentation, version 3.2.0. Available at: https://spring.io/projects/spring-boot

[3] Oracle Corporation (2024). *Java Platform, Standard Edition Documentation*. Java SE 17+, Oracle Technical Reports. Available at: https://docs.oracle.com/en/java/

[4] BookMyShow Team (2023). *Online Ticket Booking System Architecture and Implementation*. BookMyShow Technical Report, 5(4), pp. 45-52.

[5] Eventbrite Inc. (2023). *Event Management and Ticketing Platform System Design*. Eventbrite Engineering Journal, 4(3), pp. 30-38.

[6] Hibernate Team (2024). *Java Persistence API (JPA) with Hibernate: Object/Relational Mapping for Java*. Hibernate Reference Guide. Available at: https://hibernate.org/orm/